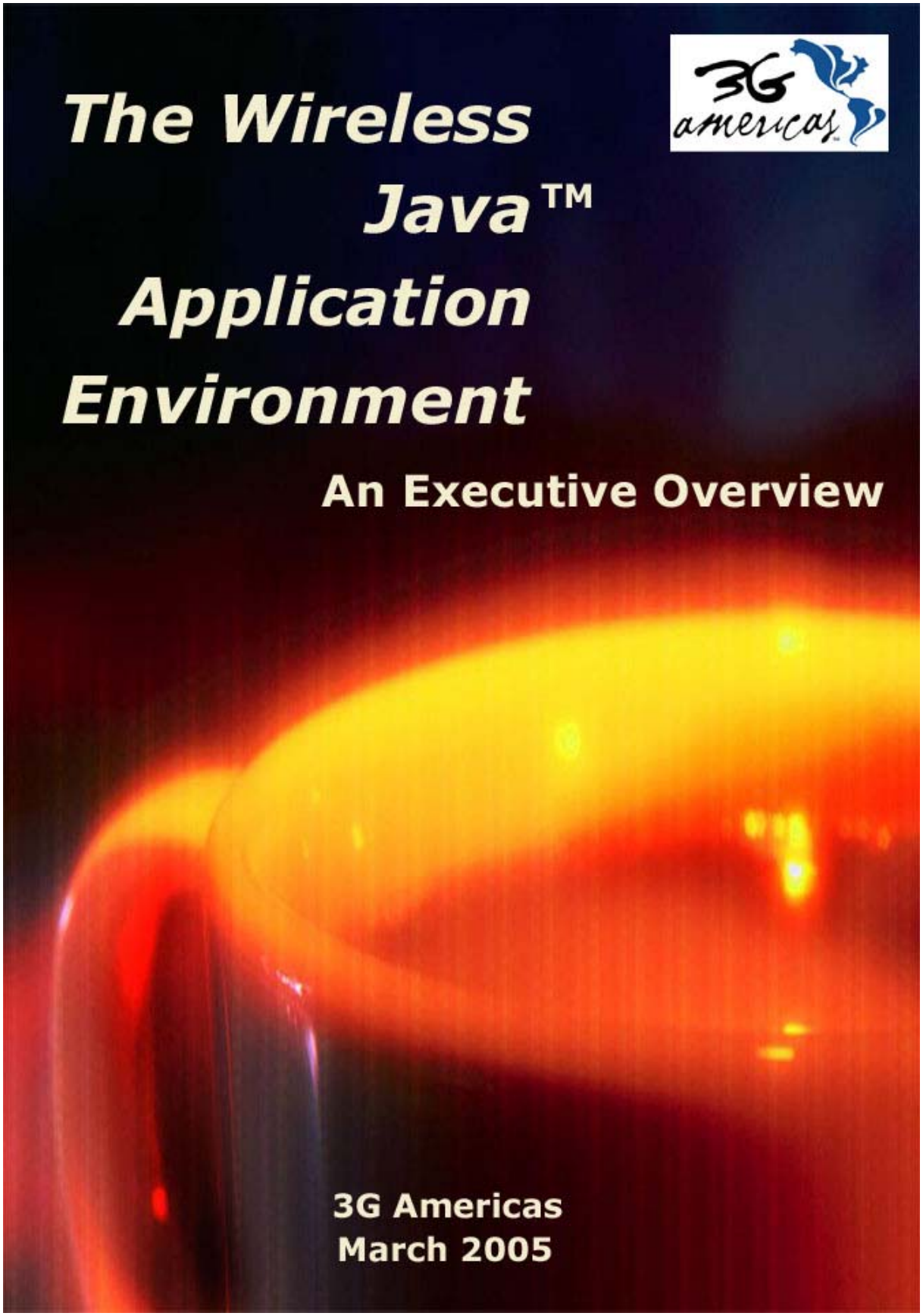
The Wireless Java™ Application Environment
An Executive Overview
3G Americas
March 2005

Table of Contents

Figures

1. Introduction

This application-environment white paper is a high-level overview of the Java™ technology ecosystem as a complete, flexible, and technologically competitive environment. It covers:

- Java standards development
- Role of technology suppliers
- Desired improvements to testing and certification, to ensure a consistent device environment and flexible applications options
- Developers' support and developers' tools
- Evolution of carriers' solutions and offers through any number of deployment scenarios

2. Executive Summary

Projected to reach *1.5 billion consumers* by 2007, Java technology continues to drive the adoption of mobile data services. Research by Evans Data Corp. found that 40 percent of all mobile handsets have Java platform capabilities. Indeed, in the five years since its inception, the Java 2 Platform, Micro Edition (J2ME™ platform) has achieved an installed base of more than 574 million devices and over 399 unique models. The research directly attributes J2ME technology's growth to the wide-spread adoption of J2ME platform-capable devices in the consumer marketplace.

More than *110 operators* worldwide, spanning GSM, CDMA and IDEN networks have already deployed Java Services. This has been possible because Java technology is not dependent on a specific network infrastructure or business model. Such flexibility is making Java technology the de facto standard for developing and delivering mass-market mobile applications.

Java technology's download services have opened up many new business opportunities for those in the Java technology ecosystem. For example, operators enjoyed $1.4 billion in revenue in 2003 as a result of offering Java services, and this number is expected to grow to $15 billion in revenue by 2008 according to ARC. Operators, content providers/owners, content aggregators, handset manufactures, and application developers have all benefited from the thriving Java technology ecosystem.

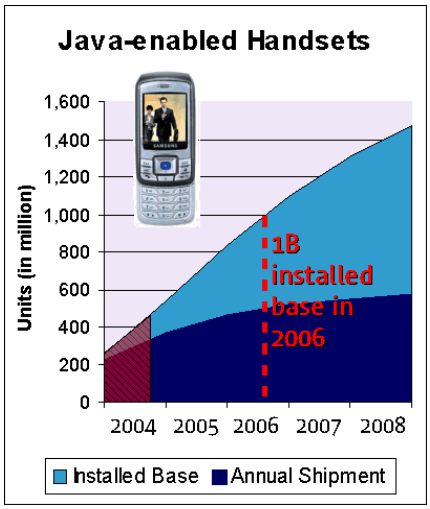

Figure 1: Growth of Handsets with Java Technology

2.1. Adaptable

As the needs of the marketplace change, the J2ME platform adapts. Instead of being optimized for a particular company's needs, the technology evolves through the cooperation of the members of the Java Community Process (JCP). JCP members span the entire wireless industry, and no single participant holds a dominating position. New members are encouraged to join.

The JCP's Java Specification Requests (JSRs) detail changes to the Java platform. There are 68 JSRs for J2ME, led by 19 specification leads. All JCP activities are overseen by its executive committee (EC), which the members vote in each year.

2.2. Secure

Even as the adoption of Java technology continues to grow rapidly, and the platform evolves, it's worth noting that security is a critical priority in the wireless Java application environment. The J2ME platform's security domain model is designed to provide maximum protection, as well as the flexibility and extensibility to support different operators' security implementations.

2.3. Popular

Over 4 million developers create J2ME software. These developers use a wide variety of tools to build the compelling applications we enjoy today. Java technology has the most device applications available, and the numbers just keep getting better and better.

The popularity of the J2ME platform is not only due to the vast number of available applications. The J2ME platform itself has been optimized for mass adoption. There are a number of deployment scenarios and flexible implementation options, including fully hosted models. The scenarios include complete turnkey solutions for identifying, certifying, distributing, and settling content sales.

The health of the Java community is a direct result of the involvement by all of the members of the Java Community who make this possible.

3. Overview of the Java Environment

As the most widespread application development platform in the mobile industry, the Java platform provides new mass-market business opportunities and applications. Launching new mobile services based on Java technology (Java services), such as collaborative end-to-end applications and those that access consumer and business content, increases network traffic and encourages consumers to buy premium services.

The Java platform is open, widely used, robust and secure. It is familiar to millions of software developers, over 4 million by current industry estimates. Java runtime environments are used in a wide range of hardware environments, from mainframes to desktops, to more than 574 million Java-enabled handsets. This ubiquity allows enterprise and desktop software developers to extend their offerings into the mobile market, using mainly their existing tools and expertise.

3.1. Java Offers New Business Opportunities

Mobile Java platform-based applications (Java applications) offer new capabilities to both consumers and enterprise customers, enhancing the capabilities of their mobile devices for collaboration, entertainment and mobile computing. Creating and providing these capabilities opens new business opportunities.

3.1.1. Opportunities for Mobile Operators

With more than 574 million Java handsets, and a well-established developers' community, Java technology offers mobile operators new ways to boost revenues and differentiate themselves. Differentiation can increase customer loyalty, capture new recurring traffic, and generate subscription revenues.

Offering compelling, continuously refreshing Java content can lead to recurring visits and frequent information deliveries. The content can vary in interest from mass-market, such as news and business information, to small communities, such as a fan club of a popular hip-hop music group.

To minimize start-up costs, operators can build on existing technical assets to develop over-the-air distribution and payment, and can rely on their customer knowledge, channels, and brand to market Java services to their subscribers.

By launching Java services in cooperation with content aggregators, service providers, application developers and content owners, operators can gain attractive revenues from increased traffic as well as a share of the content revenues.

3.1.2. Opportunities for Application Developers

For skilled application developers, the mobile industry offers a market with huge business potential. Although today, mobile content consists of simple ring tones, logos, games, consumer and business applications, the rapidly growing number of Java devices opens new markets for advanced and higher-valued Java applications.

3.1.3. Opportunities for Service Providers and Aggregators

Java services provide a revenue growth path for service providers and aggregators. In many countries, service providers and aggregators have successfully partnered with operators to introduce Java services such as premium SMS. Downloadable Java applications, such as games, also go well with current product offerings.

3.1.4. Opportunities for Media Content, Trade, and Financial Services Companies

Media companies can provide applications that let customers track and view news, weather, or other information on selected areas of interest. Financial services can provide stock tracking and trading applications. Supermarket chains can distribute free client software that automatically downloads promotions to the Java device. The possibilities are endless.

3.1.5. The Downloadable Java Applications Business

The downloadable Java application has been the most widespread business model so far. Initially based mainly on applications like games and animated screen savers, it is now expanding into more utility and business-oriented applications.

Adding more pricing schemes to the traditional pay-per-download, such as subscriptions, can generate recurring revenue streams and build longer customer relationships. Subscriptions tend to be more profitable over the long term.

3.1.6. The Connected Java Applications Business

Advanced offerings can generate recurring revenue streams by serving Java applications. These revenue streams are more versatile - there is a fee for the application itself and for using the application over the network. Examples are:

- **Network-assisted games:** Many networking features can be added to a Java game application. Features such as a high-score service create loyalty to the service and a community among the users. Additional downloadable features such as game levels or animations can create new revenue streams.
- **Infotainment services:** Some interactive information and entertainment services, such as stock quotes, use SMS or WAP. A Java application can be another delivery mechanism, creating user loyalty and reducing churn.

3.1.7. Boosting Existing Services with Java platform-based User Interfaces

Java applications can enhance existing text or WAP-based applications with an attractive graphical user interface. For example, SMS or WAP-based mobile game services are widely used in many countries and generate substantial revenue. A customer's experience can be substantially improved with a graphical user interface, and more positive user experiences generate better revenues.

3.2. Java Technology

The Java platform consists of both a programming language and an execution environment that can run on many operating systems. Java technology's proven scalability and portability enables it to be on platforms from large enterprise servers and PCs to high-end and mass-market mobile devices.

The scalability of Java technology, and the standardization of the language across platforms, allows developers to leverage their skills across the range of Java programming environments. These environments are Java 2 Enterprise Edition (J2EE™ platform) for enterprise servers, Java 2 Standard Edition (J2SE™ platform) for PCs, J2ME platform for small mobile and resource-constrained devices, and Java Card™ for (U)SIM cards. This paper covers J2ME technology.

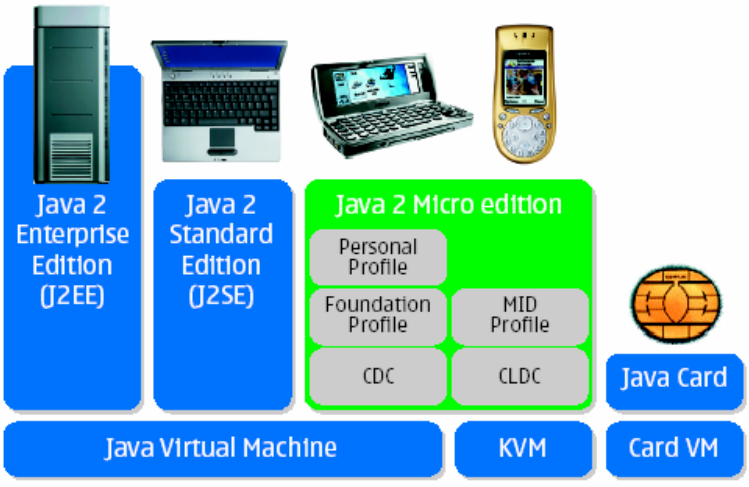

Figure 2: Java End-to-End View

3.2.1. Learn Once, Write Anywhere

The original battle cry for Java technology was “write-once, run-anywhere.” The exception is J2ME, which has extensions for small, memory-constrained devices. Another description might be “learn-once, write-anywhere,” because, for the most part, each Java platform is a subset of the J2EE platform. After learning it, a developer can create applications for any platform, after understanding the platform’s restrictions. This provides a very large pool of enterprise developers who can develop integrated end-to-end systems, not just individual applications.

Figure 3 shows how the Java environments fit together.

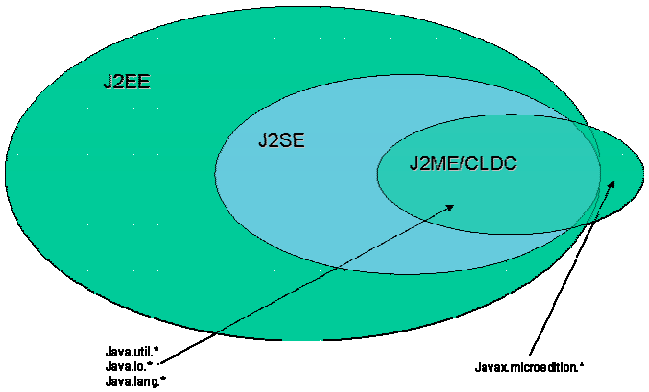

Figure 3: Java Platform Distributions

3.2.2. Feature Sets Sized for Different Devices

The J2ME platform is not a single specification. It is a collection of technologies and specifications designed for different parts of the small device market.

The core of the J2ME platform is two different *configurations*: Connected Device Configuration (CDC) and Connected Limited Device Configuration (CLDC). A configuration defines the core libraries and the virtual machine capabilities. CLDC is for low-cost, portable devices such as mainstream mobile phones. CDC is for more advanced mobile devices.

Profiles are on top of configurations and define the required functionality and application programming interfaces (APIs) for a specific device category. There are two key profiles for the J2ME platform: the Mobile Information Device Profile (MIDP) and Personal Profile (PP). MIDP is a profile for CLDC. It is for portable devices with communication capability, such as mobile phones and it defines functionality such as the user interface, persistent storage, and networking. PP is a profile for CDC for high-end portable devices and PDA’s.

J2ME also has *optional packages* on top of profiles; they extend the functionality of J2ME. Optional packages are developed by the JCP, thus J2ME can evolve as the needs of the industry change.

Figure 4 shows the Java platform architecture, including configurations, profiles, and optional packages.

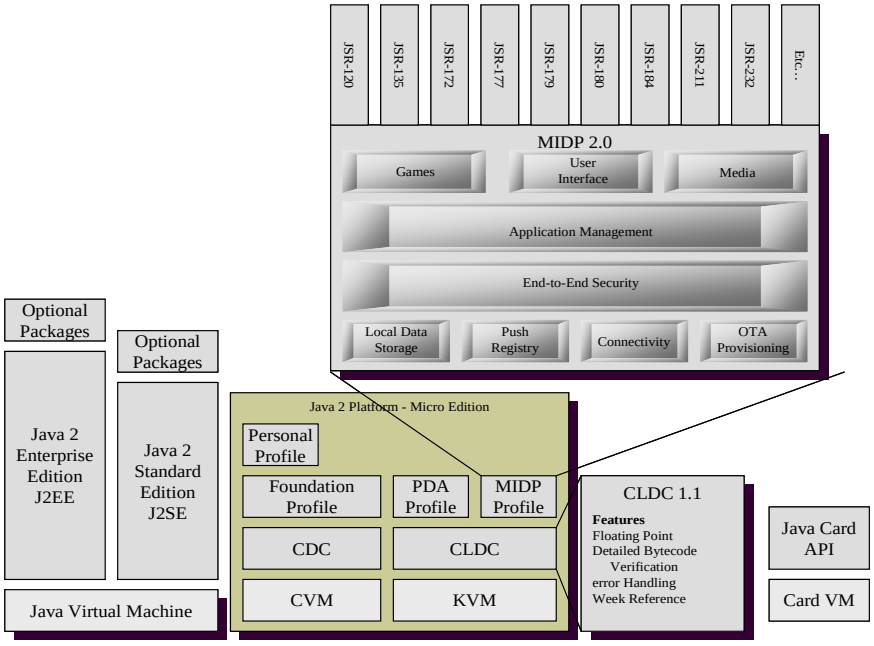

Figure 4: Java Platform Architecture

3.2.2.1. Optional Packages

There are two kinds of optional packages. One type is for a particular device, and possibly market segment, but typically not for the mass-market. The second type attempts to create consistency and foster adoption by consumers and developers. Packages of the second type are mass-market packages.

Some mass-market optional packages are based on device capabilities (without being tailored to a specific device). For example, there is an optional package for Bluetooth. This package would not be put on devices that are not Bluetooth enabled; thus reducing the footprint of the Java platform and saving device resources.

Figure 5 shows a stack with optional packages. The J2ME platform architecture allows both the operator and manufacturer to make the best use of a mobile device's scarce resources by fitting the Java environment to the device's capabilities and constraints.

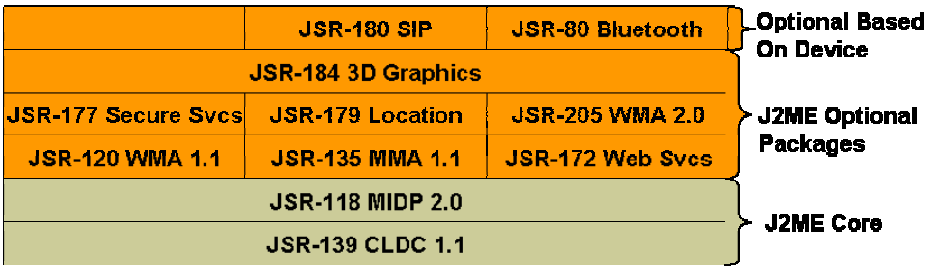

Figure 5: Stack for the Mobile Java Platform

3.2.2.2. Organizing MIDP: JSR 185

Because it was not clear how the various technologies associated with the MIDP profile work together to form a complete handset solution, JSR 185, Java Technology for the Wireless Industry, was formed in May 2002. Its charter was to recommend a roadmap for the inclusion of specific optional packages, to create a more consistent solution. The group that worked on this JSR (the *expert group*) had members from all aspects of the wireless industry, including operators.

3.2.3. MIDP 2.0

MIDP 2.0 is an important convergence point for the industry. It brings together the fragmented Java implementations, which improves the economy of scale in deploying Java applications across devices. MIDP 2.0 features:

- Radically improved graphics and media features
- Multipurpose networking features
- Advanced security features
- Backward compatibility for running applications based on MIDP1.0

In addition to MIDP 2.0, the following packages are part of the J2ME environment:

- JSR 139 - Connected Limited Device Configuration (CLDC) 1.1
  http://jcp.org/aboutJava/communityprocess/final/jsr139/index.html
  Core platform libraries, security and networking
- JSR 184 - Mobile 3D Graphics API for J2ME
  http://jcp.org/aboutJava/communityprocess/final/jsr184/index.html
  Supports 3D graphics
- JSR 179 - Location API for J2ME
  http://jcp.org/aboutJava/communityprocess/final/jsr179/index.html
  Provides access to location information from the J2ME environment
- JSR 120 - Wireless Messaging API
  http://jcp.org/aboutJava/communityprocess/final/jsr120/index2.html
  Enables J2ME applications to receive SMS-based notifications
- JSR 135 - Mobile Media API
  http://jcp.org/aboutJava/communityprocess/final/jsr135/index2.html
  Provides access to rich media capabilities from the J2ME environment
- JSR 82 - Java APIs for Bluetooth
  http://jcp.org/aboutJava/communityprocess/final/jsr082/index.html
  Allows access to a Bluetooth network from the J2ME environment
- JSR 75 - PDA Optional Packages for the J2ME™ Platform
  http://jcp.org/aboutJava/communityprocess/final/jsr075/index.html
  PIM and File system access from within the J2ME environment
- JSR 180 - SIP API for J2ME
  http://jcp.org/aboutJava/communityprocess/mrel/jsr180/index.html
  Access to SIP functionality from within the J2ME environment
- JSR 177 Security and Trust Services API
  http://jcp.org/aboutJava/communityprocess/final/jsr177/index.html
  Provides access to security and signing mechanisms for J2ME applications
- JSR 229 - Payment API
  http://jcp.org/aboutJava/communityprocess/pr/jsr229/index.html
  Allows applications to obtain payment for an application feature
- JSR 172 - J2ME Web Services Specification
  http://jcp.org/aboutJava/communityprocess/final/jsr172/index.html
  Makes web services accessible from within the J2ME environment
- JSR-211 - Content Handler API
  http://jcp.org/aboutJava/communityprocess/pfd/jsr211/index.html
  Allows dynamic support for handling new content types on a device

There are even more packages available. For more information, see http://jcp.org.

3.2.4. Security and Privacy

MIDP 2.0 has a security domain model designed to protect the user and operator from malicious attacks on a mobile device. This domain model is extensible and gives operators flexibility in their security implementations.

MIDP 1.0 had minimal functionality and used a sandbox architecture, which minimized the impact of attacks on the device and its applications. MIDP 2.0 extends the ability of Java applications to integrate into the Internet architecture. With these new capabilities, operators must use the MIDP 2.0 security domain model to develop security policies that protect both the network and its customers. This is critical in keeping Java a very secure environment, while allowing operators to control the level of access they grant to external entities developing compelling applications for their customers.

The MIDP 2.0 security model is extensible enough for operators to allow access to sensitive services for some applications, while preventing unpredictable behaviors and actions by restricting access to others. The flexibility comes from matching each application with a security domain. Each domain enforces a specific security policy that limits access. For instance, if an application belongs to a third-party domain, it can access only the third-party domain's accessible features.

An application's domain is determined by the certificate attached to the application. The certificate must use industry-standard public-key infrastructure (PKI) techniques. When the system loads the application, it verifies the certificate, matches it to the domain, and runs the application with only the domain's permissions.

The Java Community Process developed a best practice's document that provides a minimum set of domains that each GSM operator should support. They are:

- Manufacturer Domain
- Operator Domain
- Trusted 3rd Party Domain
- UnTrusted Domain

3.3. Java Standards

As an open and non-proprietary industry standard, the Java platform must be standardized while remaining open to improvements and innovation. This is assured through the *Java Community Process* (JCP), which includes representatives from many system and telecom companies. The community process assures comprehensive review and consensus and encourages continual improvement.

3.3.1. Java Standards Overview

The Java technology community, through the JCP, drives Java platform standards. The JCP, which consists of members, licensees, and non-members, develops and maintains the standards, and upholds the cross platform philosophy of Java. Figure 6 shows the JCP hierarchy.

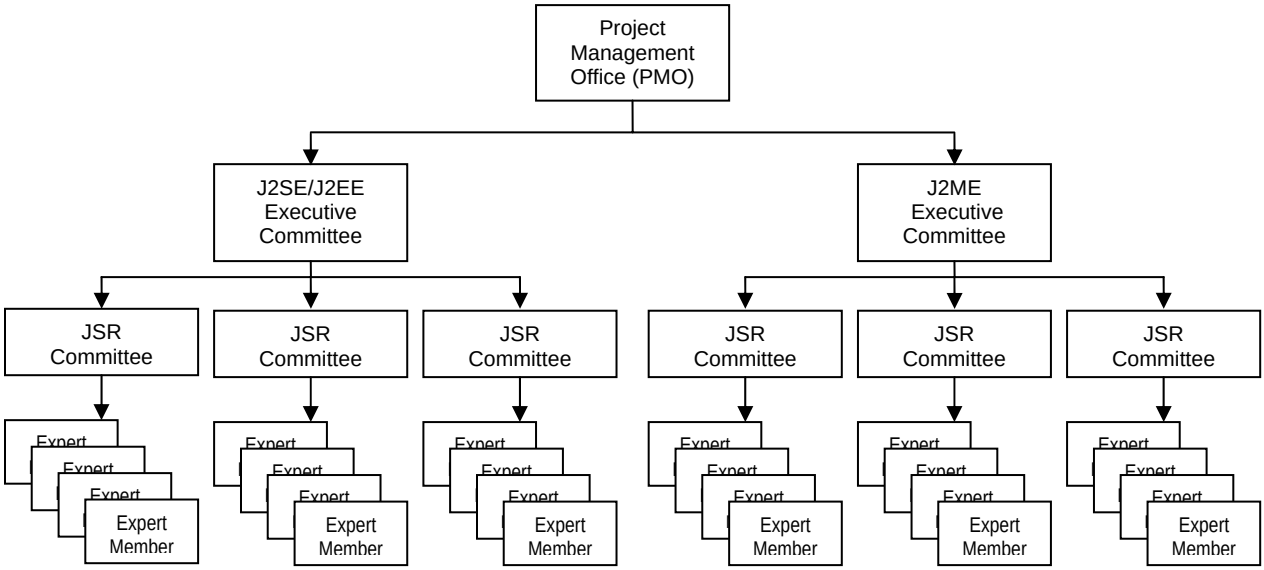

Figure 6: JCP Hierarchy

The governing body of the JCP is the Project Management Office (PMO), which both administers the JCP and chairs the Executive Committee (EC). The EC is the group of JCP members guiding the evolution of Java technology. To create or change a Java standard, a JSR needs to be submitted to the PMO specifying what the proposed standard will accomplish. The JSR contains the proposed and final specifications for the Java platform.

At any given time there are numerous JSRs moving through the JCP process. When the EC approves a JSR, they assign a committee member recruit and staff the JSR committee or Expert Group (EG). The EG crafts the specification for presentation and adoption by the EC.

The following platforms and technologies are critical to operators. They help drive these technologies through the JSR process.

- **J2ME technology**
- **J2EE technology**
- **XML:** Extensible markup language is a universal syntax for describing and structuring data independent from application logic
- **JAIN**: Java APIs for Integrated Networks focuses on emerging network protocols and architectures that are driven by convergence of traditional telecommunication and IP networks

3.3.2. Executive Committees

The EC is an elected group of JCP members that guide the evolution of Java technology. The EC represents both major stakeholders and a representative cross-section of the Java Community. There are two ECs: the SE/EE EC, which is responsible for the J2SE and J2EE platform specifications, and the ME EC, which is responsible for the J2ME platform specification.

Each EC is expected to:

- Select JSRs for development
- Approve draft specifications for public review
- Give final approval to completed specifications, as well as their associated reference implementations (RIs) and Technology Compatibility Kits (TCKs)
- Decide appeals of first-level TCK test challenges
- Review maintenance revisions, possibly requiring some to be in a new JSR
- Approve transfer of maintenance duties between members
- Provide guidance to the PMO as it handles the day-to-day running of the JCP

3.3.3. Specification Process

The entire JCP process is described at http://jcp.org/. There are four major steps in the JSR process:

1. **Initiation**: Community members initiate a specification for the responsible EC to approve.

2. **Early draft**: An EG develops a preliminary draft of the specification, which the community and the public (anyone with an Internet connection) review. The expert group uses review feedback to refine the specification.

3. **Public Draft**: Anyone can review this new draft of the specification. Again the EG uses review comments to revise the document. At the end of this review, the EC decides if the draft should proceed. If the EC approves, the leader of the EG sees that the RI and TCK are completed before sending the specification to the responsible EC for final approval.

4. **Maintenance**: The completed specification, RI, and TCK are updated in response to ongoing requests for clarification, interpretation, enhancements, and revisions. The responsible EC can review all proposed changes to the specification and indicate which ones can be carried out immediately and which will require the specification to be revised by an expert group. Challenges to one or more tests in a specification's TCK are ultimately decided by the responsible EC if they cannot be otherwise resolved.

3.3.4. The Future of Java Specifications

J2ME will continue to address the needs of mobile consumer users. It will also begin to address the needs of enterprise users and corporate IT managers if there is a tight integration built between J2ME, J2EE, and Web Service technologies. A common set of client and server platform APIs would help accomplish this goal. In such a future, enterprise users will access their corporate intranets, databases and other corporate services more easily and securely using their J2ME technology mobile devices.

4. Java Technology and the End-to-End Experience

The Java environment provides the operator an end-to-end environment for discovering and delivering content tailored to a user's personal preferences and device capabilities. The entry point into the discovery and download process is a presentation server. A presentation server works with a download server to provide content discovery, content delivery, and billing.

Content discovery matches the user's preferences and device capabilities with the contents of the data repository. The presentation server shows these matches. Delivery to the mobile device happens after the user selects the desired content. After delivery, the download server charges the customer for the content. See JSR-124, Client Provisioning Specification, for more information.

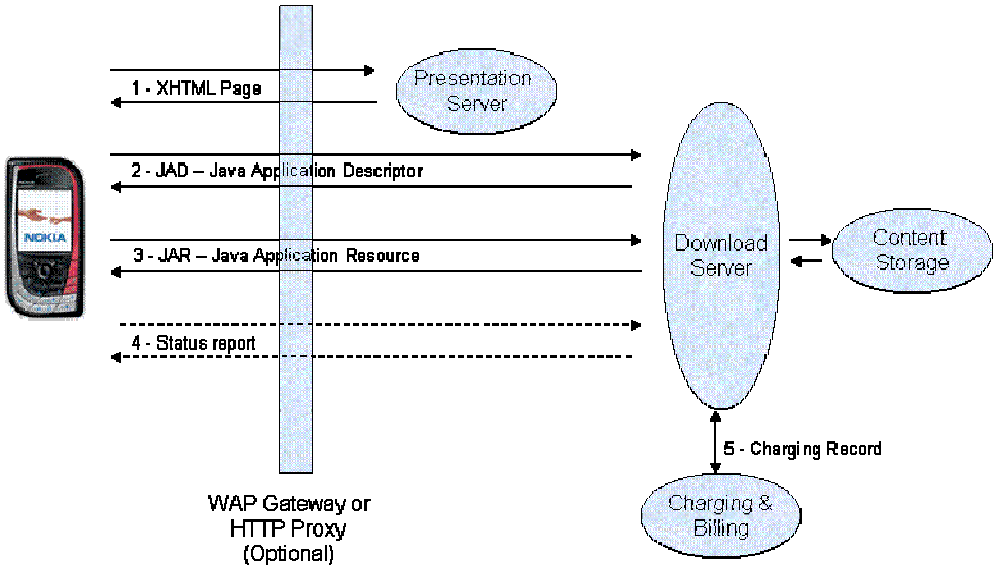

Figure 7: Download Architecture

Figure 7 describes the end-to-end network elements and process. In this case, the mobile user browses color XHTML (WAP2.0) pages and discovers an exciting Java game. The user decides to download the game and clicks the download link, which points to a download server. The download server delivers the Java Application Descriptor (JAD) file then, after the device verifies that it can accept the application, delivers the JAR file. The phone sends a status report after it installs the content, which allows reliable billing.

The building blocks for downloading Java applications over the air, in addition to the presentation and download servers, are a WAP gateway or HTTP proxy, and the HTTP protocol or corresponding WSP protocol in the WAP stack.

4.1. End-to-End Elements

Operators need to begin developing the infrastructure and business relationships necessary in creating and delivering next generation services. This section describes business models that operators can use. The models are basic, and there are hybrids of each. Operators can consider which type of model fits their business. Choosing one model does not mean being stuck with it.

The Java platform and the following business models provide flexibility. The platform's modularity, and openness to both integrated and component technologies, is an inherent benefit of the Java platform, as opposed to alternative closed solutions. This flexibility allows operators to provide Java applications and services in ways that can evolve with its business models.

4.1.1. Operator-as-Pipe

"Operator-as-Pipe" is the simplest end-to-end solution; it makes the operator a data pipe with little or no involvement in the selection of content. Content providers retain maximum freedom. Customers might face content discovery challenges, but the operator or content aggregator can alleviate the challenges by designing creative and relevant content discovery tools. Data transfer fees provide revenue.

The Operator-as-Pipe deployment is extremely straightforward. It is useful in early deployment phases and for those operators that want to outsource content management and billing. The operator controls its network and customers, and can negotiate specific business arrangements with content providers, as needed.

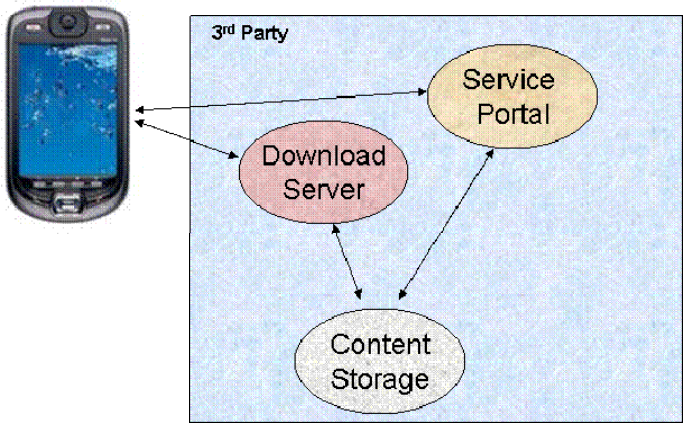

Figure 8: Operator-as-Pipe

4.1.2. Operator-as-Vendor

In the "Operator-as-Vendor" model, the operator manages all of the details, including customers, communication, billing, and content offerings. Content providers either respond to the content development requests of the operator, or find that the operator more carefully manages their independently developed offerings. Customers might have a more cohesive experience, given the operator's centralized management.

This solution might be most suitable for an experienced operator with a well-defined strategic need to carefully manage their content experience. However, with this level of control comes the need to manage the infrastructure and the overall user experience at a level higher than the other deployment models.

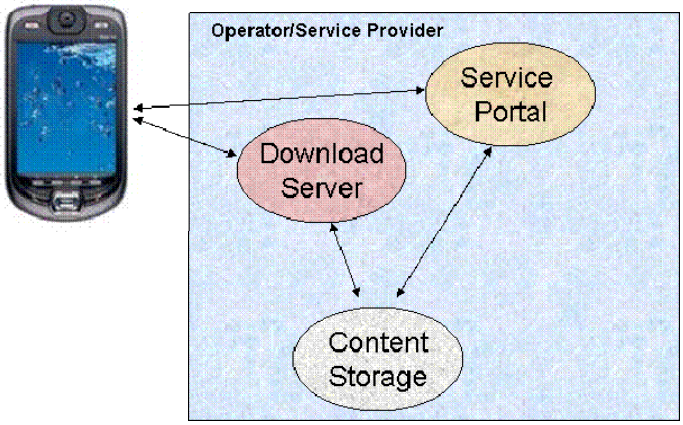

Figure 9: Operator-as-Vendor

4.1.3. Operator-as-Broker

"Operator-as-Broker" is a hybrid model. As a broker, the operator actively enables, manages, and brokers the relationship between the customer and content provider. An example of this is the successful iMode content deployment model, in which the iMode network delivered large numbers of customers to content providers, and gave content providers simple mechanisms for identifying and billing customers through the purchasing transactions brokered by DoCoMo. The operator typically captures a small percentage of the brokered transaction's revenue.

This model gives the operator wide leeway in defining the marketing mechanisms, the relationship of the customer and content provider, and their own relationships with content providers. The content providers may be fully independent content creators, or create specific content in response to operator requests.

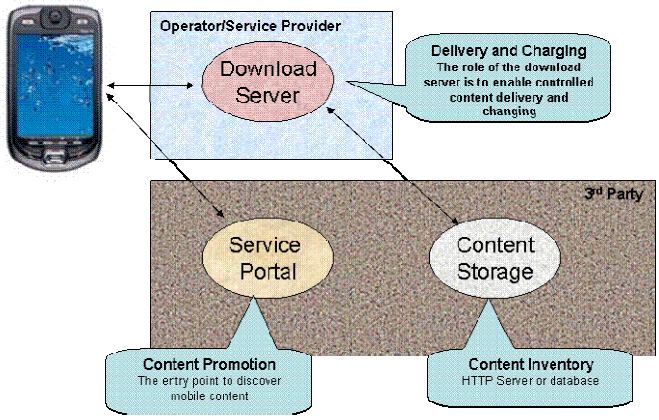

Figure 10: Operator-as-Broker

4.1.4. Future Evolution

These models give operators many content, infrastructure, and content aggregation options. The many end-to-end components also mean that the Java platform offers the freedom to evolve and change solutions over time to adapt to customer and competitive needs.

4.2. Using Content Providers

Content providers give a catalog of products to consumers. They can sell directly using operators' pipes, or intermediaries like message aggregators, service aggregators, or using operators' distribution channels.

Some content providers might develop content, like games or music, using their own intellectual property or a third party license. These content providers are often referred as content developers.

Operators are increasingly reluctant to deal with a large number of small and medium-sized content developers. Instead, content aggregators qualify and filter the developers' content, or content publishers aggregate and develop content at the same time.

When content providers use a business-to-consumer model, they are responsible for the quality of their product and support of their customers. When they use a business-to-

business approach, they share quality and support responsibilities with vendors. Content providers generally try to certify their content on a majority of available devices. They also maintain and operate any server-side component of their products such as a mail server for an email application, a content feed for a news reader, and so on. These models are comparable to the online business with Portal direct to consumer access and content syndication.

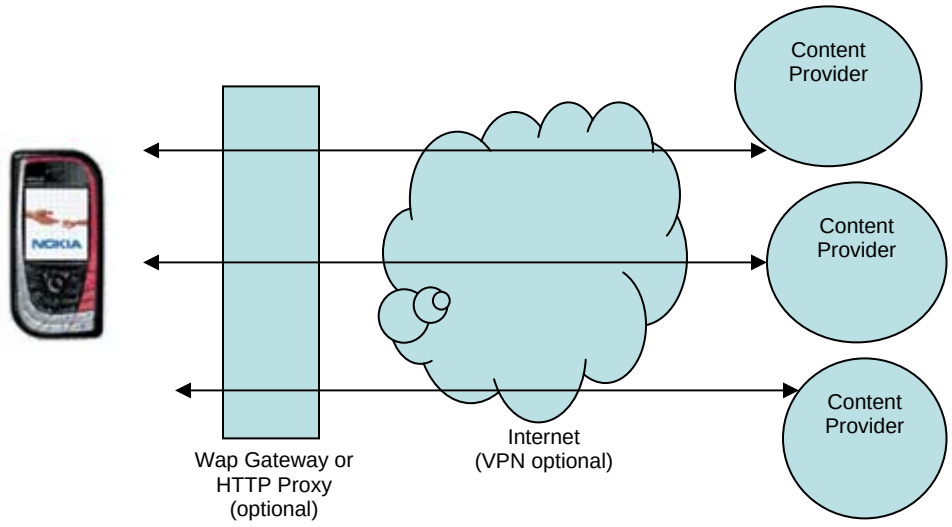

Figure 11: Using Content Providers

Content providers might securely connect to the operators' infrastructure or service aggregator to enable the delivery of their applications to consumers and to let them use all wireless data possibilities to offer the most compelling experience. Therefore, operators can think of content, in general, as active after it is downloaded -- Java applications and games may connect to databases, chat services, and so on. They may need to use TCP/UDP IP, HTTP, and HTTPS connections, and SMS entities, SMS brokers, MMS relays, and billing systems.

4.3. Testing and Certification

Testing and certification ensures that devices satisfy the technical requirements set forth by this working group. The requirements specifically address areas where quality and consistency can be improved through carrier clarifications of ambiguous or unstated requirements within the Java standards. Some areas outside the scope of standards are also expected to be the target of carrier requirements.

Note that these specification issues are an intentional outcome of the Java standards process. The process ensures that a specification imposes a good level of standardization while leaving room for variations that allow the standard to be effectively applied in current and future unanticipated environments. The freedom for specific markets to define their own narrower interpretation of the standards has always existed, although it is not always exercised.

For example, a JAD file is a descriptive file downloaded to the mobile device, which describes and points to the MIDlet that is to be downloaded. The file defines any number of parameters used at runtime. The standards set no requirement on the minimum desired size of the file or the allowed length of words and values within the file. This forces Java implementations to choose arbitrary (and possibly different) capacities for acceptable JAD files, forces developers to be overly conservative in their JAD file construction, and may still lead to unexpected incompatibilities as an operator introduces

a new device into their portfolio only to find that said device has capacities that are insufficient to handle existing JAD files[1].

The scope of the testing function is limited to the working group's stated requirements. Broad requirements related to existing standards certification (such compliance to the MIDP 2.0 standard itself) is out of scope and more appropriately addressed by available industry testing tools, a JCP certification process, and individual carrier requirements.

An initial activity of this working group will be to assign test development, execution, and certification responsibilities. This process is relatively open-ended at this time and will require an assessment of required resources and entities suited to the distinct tasks of authoring tests, executing tests, and issuing a declaration of certification. The requirements will emerge from this working group, but the actual testing and certification may be conducted under the auspices of separate entities better suited to those roles.

A general methodology for testing and certification can be summarized as follows. This process would be executed once requirements have been defined.

1. Identify and aggregate testable requirements
2. Define assertions and pass/fail criteria
3. Identify testing harness or test tool requirements; collectively, the requirements are likely to require multiple harnesses or tools, although some may be relatively trivial
4. Author tests
5. Execute tests and exchange results with device supplier; this step would need to be governed by appropriate NDA policies
6. Upon satisfaction of overall pass/fail criteria, issue a device certification report

The certification declares that the device meets the requirements stated by this working group. As mentioned previously, it does not assert that all functionality regulated by other standards and all individual carrier requirements are implemented correctly. However, it does rein in the significant and uncontrolled variability that would exist in the absence of a testing and certification process. This working group's requirements will represent the most important issues identified by carriers. An enforced testing and certification process should deliver significant improvement.

4.4. Developers Support

The Java platform and the Java platform community have fully embraced the open source philosophy when it comes to developers support. Java tools and technology are freely available from a wide range of resources. Sun Microsystems, the creator of Java, provides a full range of tools, technology and support. In addition, developers can get tools and training from a long list of developer support programs provided by:

- **The wireless operator community:** In addition to Java training and tools, the developer can learn the techniques of a particular network architecture.
- **Handset manufacturers**: Developers can get SDKs and testing tools, along with specific information about the manufacturers' devices' capabilities
- **Hardware and software companies:** Companies like Sun, IBM, BEA, and others have considerable development and training tools available.

[1] As said in the previous section, applications may be connected to communicate with a specific server. For example, the location of the server can be used as a JAD parameter as a URL or many entries that specify things like IP address, port number, name, protocol, etc. Servers generally need to know which device model is currently in use for example to deliver the right size of content and not overload the device memory. Yet the device model is another attribute that content providers may want to add in the JAD.

- **Periodicals:** Many books and magazines are dedicated to the Java community; any of them provide a wealth of information to both new and experienced developers.
- **Continuing education:** Local community colleges and online training courses provide continuing education.

An advantage for J2ME platform developers, that other developers do not have, is that J2ME is a subset of the Java enterprise environment. Many of the same skills used at the enterprise level are used in J2ME. The developer merely needs to learn the extensions and differences necessary for the J2ME environment, a much smaller task than learning a whole new programming language. With a free SDK and a free emulator provided by handset manufacturer or Sun Microsystems, software engineers with Java platform experience or a solid background in object-oriented programming be writing MIDP applications in no time.

The design of the Java environment provides the following advantages:

- Enterprise developers with object-oriented skills can easily move to developing Java and J2ME applications
- Java developers can build true, end-to-end wireless enterprise and consumer applications using the J2ME platform on the handset and the J2EE platform in the enterprise
- Businesses can employ one skill set for developing and delivering true, end-to-end network aware applications regardless of the target deployment platform

Java platform developers are uniquely positioned to take advantage of Java technology on all deployment platforms from the enterprise all the way to the smallest terminal devices. As stated earlier, the mantra for Java is “Learn Once – Write Anywhere,” and much like any environment it is essential for the developer to test the application on target devices in order to determine not only the performance of the devices but also that any non-standard proprietary APIs from the manufacturer or operator are available. Operators should provide a detailed description of the conformance of their environment through their developer’s assistance program.

Unlike almost any other operating environment, developers are encouraged to participate in the development of the Java platform. All Java platform standards are public, available from http://jcp.org. In addition, developers are encouraged to actively participate in the development of the Java platform through participation in an EG or by commenting on the specification before it is finalized. This process allows for the best implementation available, ultimately meeting the needs of developers.

Additional resources for Java technology and tools can be obtained from:

- http://java.com
- http://java.net – the source for Java technology collaboration
- http://www.sun.com/java - downloads for Java technology and tools
- http://www-130.ibm.com/developerworks/java/ - Java technology and tools

4.5. Wireless Java Standards Governance Process

As an open and non-proprietary industry standard, the Java programming language interfaces and environments have to be standardized while remaining open to improvements and innovation. The JCP controls and regulates the standard. The JCP includes representatives from many system and telecom companies. The JCP assures comprehensive review and consensus, and encourages continual improvement.

4.6. Wireless Java Interoperability Testing

Testing and certification is essential to allowing applications to be used on a wide range of devices worldwide, on many different networks. Sun Microsystems has developed the Java Device Test Suite, which is available as a software product. There are other commercial and free validation and test tools from Nokia, Openwave and IBM.

The JCP also requires each JSR to provide a Technology Compatibility Kit (TCK).

A group of major operators has formed the Open Mobile Terminal Platform alliance, which aims to standardize the look and feel of mobile applications. It also aims to provide guidelines for Java mobile application development.

Another industry consortium, the Open Services Gateway Initiative (OSGi) is providing a container for dynamic software components. The containers, which will be compatible with the J2ME platform, define interactions between components and allow developers to remotely manage the entire application life cycle, including over-the-network deployment and updating.

5. Conclusion

Java offers new business opportunities to mobile operators. It opens up new data-related applications beyond text messaging and phone personalization downloads. Purchasing and downloading new Java applications that access consumer and business content, and end-to-end Java applications like mobile games and collaboration software, increases network traffic, encourages premium services, and allows revenue sharing with content providers.

Because Java is an open, widely used, and secure language and operating environment, it is familiar to millions of software developers who can then deploy their mobile applications on the more than 574 million Java-enabled handsets in use today. The various Java runtime environments allow it to be used in a wide range of hardware environments, from the most inexpensive consumer phone to smart phones, PDAs, up to desktops and mainframes. This allows enterprise and desktop software developers to extend their offerings into the mobile market using mainly their existing tools and expertise. By current industry estimates there are already more than 4 million trained Java developers. Java development tools are available for free from several companies.

The secure licensing of applications and content is assured through Java-integrated enhancements to the SIM-based smart card technology (JSR177), which will protect their assets from piracy.

Mobile Java applications offer new capabilities to both consumers and enterprise customers, enhancing the capabilities of their mobile devices for collaboration, entertainment and mobile computing.

The J2ME platform delivers powerful cutting-edge applications and wireless connection benefits to your cellular phone or PDA. J2ME is already inside millions of devices – it is a leading platform of choice for developing and delivering consumer tools, applications, and features. Java technology is doing well in the worldwide market.

Figure 12: The Java Platform in the Worldwide Market

In addition to the numbers, there are some additional facts worth mentioning as shown in Figure 12. Java is not just for the GSM markets. In fact, Java has been well adopted within the CDMA market as well. The first CDMA carrier to deploy Java services was LG Telecom back in 2000. Since then, Java has been taking on a stronger presence across the globe including China Unicom, Bell Mobility, Sprint PCS, SK Telecom, LG Telecom, KTF, Reliance, Telus Mobility and others all offer Java services to their customers. In fact, the entire CDMA market in Korea will soon deploy Java as a core component of WIPI, their national mobile Internet standard.

The Java brand is recognized throughout the world for developing and offering rich, compelling content. This paper has explained how the Java technology is a leading element in the mobile ecosystem in the development and delivery of enterprise and consumer tools, applications, and features.

Appendix A. Glossary

Term or Acronym	Definition
API	Application Programming Interface Interfaces, classes, method, and fields available to a developer
CDC	Connected Device Configuration Core APIs for more advanced mobile devices
CLDC	Connected Limited Device Configuration Core APIs for resource-constrained mobile devices
EC	Executive Committee within the Java Community Process
J2EE	Java 2 Enterprise Edition A programming environment for enterprise customers
J2ME	Java 2 Micro Edition A programming environment for mobile devices
J2SE	Java 2 Standard Edition A programming environment for personal computers
JAR	Java Archive File See http://java.sun.com/j2se/1.5.0/docs/guide/jar/jar.html
JAD	Java Application Descriptor See http://jcp.org/en/jsr/detail?id=37 and http://jcp.org/en/jsr/detail?id=118
JDTS	Java Device Test Suite Tool to simplify the Java platform quality assurance process
JSR	Java Specification Request Document to propose adding new functionality to the Java platform
JVM	Java Virtual Machine
KVM	K Virtual Machine Java Virtual Machine for the J2ME environment
MIDP	Mobile Information Device Profile See http://jcp.org/en/jsr/detail?id=37 and http://jcp.org/en/jsr/detail?id=118
OTA	Over-the-Air Downloading applications to a mobile device from a network
RI	Reference Implementation Implementation of a JSR that demonstrates the specification
TCK	Technology Compatibility Kit Tests that show whether an implementation complies with a JSR

Appendix B. JSRs

Several JSRs were referred to briefly in Section 2.2.3. This appendix has more information about each of those JSRs, including its URL and an explanation of its importance.

B.1. JSR 139 - Connected Limited Device Configuration 1.1

http://jcp.org/aboutJava/communityprocess/final/jsr139/index.html

JSR 139 is the most current specification of CLDC, a core API for the J2ME framework. It is the foundation for MIDP 2.0, and includes basic security and networking APIs.

B.2. JSR 184 Mobile 3D Graphics API for J2ME

http://jcp.org/aboutJava/communityprocess/final/jsr184/index.html

This JSR provides APIs to display animated 3D graphics in real time. Applications can mix-and-match an immediate-mode interface for individually rendered objects, and an easy-to-use scene graph structure. The JSR also defines a file format for efficiently storing and transferring data such as textures, scene hierarchies, and animation key-frames.

B.3. JSR 179 Location API for J2ME

http://jcp.org/aboutJava/communityprocess/final/jsr179/index.html

JSR 179 enables an application to determine a device's location. A location contains geographical coordinates (latitude, longitude and altitude), an indication of the accuracy of the geographical coordinates, and a timestamp. It might also contain the device's speed and course, as well as textual address information such as a street address. JSR implementations can use any location mechanism or mechanisms, such as GPS or Bluetooth Local Positioning. Applications that use the API can request a single location or can be periodically updated with new location information.

The JSR also includes a database of landmarks. A landmark is a known physical location that is associated with a name. Users can store commonly used locations, such as their office and home, in the database. For consistency, all Java applications share the landmark database.

B.4. JSR 120 Wireless Messaging API

http://jcp.org/aboutJava/communityprocess/final/jsr120/index2.html

JSR 120 provides access to a device's messaging capabilities. Applications can send SMS messages, and can receive SMS notifications sent to Java applications that register as recipients. The Java platform is capable of starting an application, or awakening it from an idle state, to receive the message.

This JSR works with both MIDP 1.0 and MIDP 2.0 security mechanisms. The SMS bearer is treated similarly to another networking interface (such as UDP).

B.5. JSR 135 Mobile Media API

http://jcp.org/aboutJava/communityprocess/final/jsr135/index2.html

JSR 135 provides APIs to handle media ranging from simple tone generation to loading a playing video content. It includes mechanisms to capture or read data from a source (such as a file, the device's audio-input, or a streaming server) into a media processing system. It could also be used to process the data (parse or decode it, for example). Finally, the JSR has APIs to render data to an output device such as an audio speaker or video display.

B.6. JSR 82 Java APIs for Bluetooth

http://jcp.org/aboutJava/communityprocess/final/jsr082/index.html

JSR 82 provides access to a Bluetooth network, but not for accessing Audio or Telephony Bluetooth functionality. It is designed to work over a variety of Bluetooth profiles, to securely enable the following functionality:

- Register services
- Discover devices and services
- Establish RFCOMM, L2CAP and OBEX connections

It is designed to enable the creation of ad-hoc peer-to-peer networks, and to provide short-range wireless networking to kiosk or vending functionality.

B.7. JSR 75 PDA Optional Packages for the J2ME™ Platform

http://jcp.org/aboutJava/communityprocess/final/jsr075/index.html

JSR 75 provides the FileConnection package, to provide access to file systems on devices and/or mounted memory cards, and the PIM package, to provide access to *Personal Information Management* (PIM) data on J2ME platform devices. PIM data is defined as information included in address book, calendar application, and to do list applications.

An implementation of this JSR can support file system connectivity through the Generic Connection Framework, if the target device has the necessary underlying support. It can open connections to file systems on memory cards or in a device's memory, depending on device and operating system limitations. The types of memory cards that could be supported are:

- SmartMedia card
- CompactFlash® card (Registered trademark of SanDisk Corporation)
- Secure Digital card
- MultiMediaCard
- Memory Stick® (Registered trademark of Sony Corporation)

For the PIM package, an implementation can provide access to PIM databases that are resident on the device, or are at well known locations, such as SIM cards attached to the device or in PIM databases created and managed by the Java platform itself. Implementations, then, can provide PIM databases on devices that previously did not have them, as well as providing access to remote PIM databases from a J2ME device

B.8. JSR 180 SIP API for J2ME

http://jcp.org/aboutJava/communityprocess/mrel/jsr180/index.html

JSR 180 is enables Java applications to send and receive SIP messages. It specifies a compact generic SIP API, which provides SIP functionality in the transaction level. Implementations of the JSR enable SIP UA functionality, providing tools for implementing real-time instant messaging and presence-based applications.

B.9. JSR 177 Security and Trust Services API

http://jcp.org/aboutJava/communityprocess/final/jsr177/index.html

JSR 177 specifies a set of APIs that provides security and trust services by integrating a security element (SE). An SE has the following benefits:

- Secure storage to protect sensitive data, such as the user's private keys, public key (root) certificates, service credentials, personal information, and so on.
- Cryptographic operations to support payment protocols, data integrity, and data confidentiality.
- A secure execution environment to deploy custom security features.

J2ME applications can use these features to handle many value-added services, such as user identification and authentication, banking, and payment.

In GSM networks the SIM card operates as an SE. A device can also implement an SE, for example using embedded chips, special security features of its hardware, or only software. The JSR does not exclude any SE implementation, but optimizes some APIs for smart cards.

B.10. JSR 229 Payment API

http://jcp.org/aboutJava/communityprocess/pr/jsr229/index.html

JSR 229 defines APIs for initiating mobile payment transactions from Java applications. It does not define or imply any payment mechanism; it is generic enough to support different implementations. Implementations of this JSR can choose the technical solution providing application-initiated payments. The generic mechanism hides the payment architecture and complexity from developers. The JSR does not define any user behavior or user interface; implementations have the freedom to create the payment-related user experience.

B.11. JSR 172 J2ME Web Services Specification

http://jcp.org/aboutJava/communityprocess/final/jsr172/index.html

JSR-172 specifies two new, independent, optional packages:

- **XML parsing support:** This package enables applications to handle structured data in XML format, without having to create the structure-manipulations methods themselves.
- **Access to XML-based web services:** This package enables mobile device to access XML-based web services. It avoids defining new network protocols and formats and reuses existing standards.

B.12. Content Handler API (JSR-211)

http://jcp.org/aboutJava/communityprocess/pfd/jsr211/index.html

JSR 211 enables Java applications to be started, based on a uniform resource identifier (URI) and its MIME-type or scheme. It does this by enabling a Java application to register with the device to handle a particular MIME type; the application would then appropriately display its content.

The effect of JSR 211 will be to better integrate Java applications into the device's application management software, and give the user a more seamless transition between Java and native applications. For example, browsers, native applications, and other Java applications will be able to invoke Java applications, so that the device can support more media types and capabilities.

Acknowledgments

The mission of 3G Americas is to promote and facilitate the seamless deployment throughout the Americas of GSM and its evolution to 3G and beyond. 3G Americas' Board of Governor members include Cable & Wireless (West Indies), Cingular Wireless (USA), Ericsson, Gemplus, HP, Lucent Technologies, Motorola, Nokia, Nortel Networks, Openwave Systems, Research In Motion, Rogers Wireless (Canada), Siemens, T-Mobile USA, Telcel (Mexico), and Texas Instruments.

We would like to recognize the significant project leadership and important contributions of Ileana Leuca and Roger Mahler of Cingular Wireless as well as the other member companies from 3G Americas' Board of Governors. Additionally, we would like to recognize the significant contributions of Sun Microsystems and MForma.